
ClinProg-Bench: A Public Benchmark for Clinical-Trial Statistical Programming Automation

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Clinical-trial statistical programming—generating CDISC-conforming SDTM
2 and ADaM datasets, regulatory tables, and validation artefacts—is a high-stakes,
3 domain-specialised software task absent from existing code-LLM benchmarks.
4 We introduce **ClinProg-Bench**, a public benchmark of **250** expert-curated tasks
5 across five categories (code generation, code review, specification extraction, doc-
6 umentation, and debugging), grounded in the publicly redistributable CDISC
7 SDTM/ADaM Pilot Project and scored by deterministic, language-agnostic oracles.
8 The benchmark introduces a stateful runner contract that supports multi-agent and
9 retrieval-augmented agents, a two-channel leakage audit (SHA-256 byte overlap +
10 13-gram Jaccard, both reporting zero hits), and a dual-reviewer human audit on a
11 stratified random subset (**Cohen’s** $\kappa \geq 0.8$). Three frontier models evaluated under
12 zero-shot and fixture-grounded conditions achieve macro scores of 0.22–0.28, with
13 no model exceeding 0.35 on code generation and scoring near zero on log review
14 and debugging—establishing substantial headroom for future systems. We release
15 the corpus under CC-BY-4.0, the toolkit under Apache-2.0, and a Zenodo-archived
16 snapshot (DOI withheld for double-blind review).

17 1 Introduction

18 Clinical-trial statistical programming sits at the intersection of regulated software, domain-specific
19 data standards (CDISC SDTM, ADaM, Define-XML), and high-stakes outputs: every dataset and
20 listing in a regulatory submission is inspected by FDA, EMA, or PMDA reviewers and feeds directly
21 into approval decisions. The work is also voluminous—a single phase-III trial typically yields
22 hundreds of programs across SAS, R, and increasingly Python—and expensive: median compensation
23 for clinical statistical programmers exceeds \$120k/year and the global shortage of qualified personnel
24 is well documented.

25 Despite recent advances in code generation, applying general-purpose LLMs to clinical programming
26 surfaces systematic failure modes that existing benchmarks do not capture. Consider two repre-
27 sentative errors observed when prompting frontier models on CDISC tasks: (1) a model asked to
28 derive the SDTM AE domain populates AEBODSYS with the verbatim term instead of the MedDRA
29 System Organ Class, producing a dataset that passes functional tests but violates CDISC controlled
30 terminology; (2) a model generating an ADaM ADSL dataset assigns TRT01PN values of 1, 2,
31 3 instead of the protocol-specified 0, 54, 81, introducing a treatment-mapping error that would
32 propagate through every downstream efficacy table. These failures are invisible to general code
33 benchmarks but would trigger regulatory rejections. ClinProg-Bench is designed to expose precisely
34 this class of domain-specific correctness violations.

35 No existing public benchmark evaluates code LLMs or agentic systems on clinical-programming
36 workflows. General code benchmarks (HumanEval, MBPP, SWE-Bench, BigCodeBench) cover

algorithmic Python or open-source maintenance; biomedical NLP benchmarks (MedQA, PubMedQA) target text understanding rather than code; and pharma-internal evaluations rely on proprietary studies that cannot be released. The vacuum forces practitioners to evaluate models on hand-rolled, non-comparable test suites.

Contributions.

1. **ClinProg-Bench** (§3): 250 expert-curated tasks spanning code generation, code review, specification extraction, documentation, and debugging, all grounded in the publicly redistributable CDISC SDTM/ADaM Pilot Project (CC-BY-4.0).
2. **Deterministic scorers** (§4): five category-specific scorers using AST equivalence, set match, confusion matrix, slot fill, and patch-apply rubrics. No LLM-as-a-judge.
3. **Stateful runner contract** (§4): a language-agnostic workspace protocol that admits multi-agent and RAG-augmented systems, not just single-turn inference.
4. **Governance scaffolding** (§3): two-channel leakage audit and a Cohen’s $\kappa \geq 0.8$ dual-reviewer audit on a stratified random subset.
5. **Public release**: Apache-2.0 / CC-BY-4.0, Zenodo DOI, CI-tested on every push, and a rules-only reference baseline for scorer validation.

2 Related Work

Code-generation benchmarks. HumanEval [4], MBPP [1], and APPS [8] evaluate algorithmic Python on isolated functions; BigCodeBench [22] extends the paradigm to diverse function calls and complex instructions. SWE-Bench [10] and its agent-driven successor SWE-Agent [19] target repository-level bug fixing within open-source Python projects; RepoBench [15] benchmarks cross-file code completion; LiveCodeBench [14] evaluates on continuously updated competitive-programming problems to reduce data contamination. These benchmarks share our emphasis on deterministic scoring—unit tests, exact match, AST equivalence—but address general-purpose programming. None cover SAS or R in regulated settings, domain-specific data standards such as CDISC SDTM/ADaM [2, 3], or the documentation and validation artefacts required in regulatory submissions. ClinProg-Bench fills this gap by grounding all tasks in a real clinical-trial substrate and scoring against CDISC-conforming gold outputs.

Domain-specific code benchmarks. DS-1000 [13] evaluates data-science code generation across NumPy, Pandas, and TensorFlow with a “strict” mode that rejects syntactic variants; Spider [20] benchmarks text-to-SQL semantic parsing against gold query sets; CrossCodeEval [5] measures cross-file code completion across six programming languages; CRUXEval [7] probes code reasoning and execution understanding. These works share our emphasis on grounded, deterministic evaluation but address different domains without regulatory constraints. A key distinction is that ClinProg-Bench tasks produce outputs that must conform to externally imposed standards (CDISC controlled terminology, ADaM variable-naming conventions) rather than satisfying only functional correctness. A program that produces numerically correct output may still fail evaluation if variable names, metadata, or documentation do not comply with the submission standard—a qualitatively different acceptance criterion.

Biomedical and clinical evaluations. MedQA [11], PubMedQA [12], and MedAlign [6] probe clinical and biomedical reasoning over natural language and structured text. These benchmarks evaluate text understanding and clinical decision-making but do not assess code generation, data-standards conformance, or the software engineering pipeline that produces regulatory datasets and tables. Industry standards bodies—CDISC [2, 3], PHUSE, and the ICH [9]—define the data models, controlled terminologies, and process frameworks (GCP) that clinical programmers work within, but they provide substrate and guidance rather than benchmarking infrastructure. ClinProg-Bench bridges this gap: it is the first public benchmark that evaluates AI systems on the full clinical-programming lifecycle (code generation, review, specification extraction, documentation, debugging) against a CDISC-conforming substrate released under an open license.

Agentic benchmarks. GAIA [17] evaluates general-purpose AI assistants on tasks requiring web browsing, tool use, and multi-step reasoning; OSWorld [18] benchmarks multimodal agents in real desktop environments; WebArena [21] tests end-to-end web interaction; AgentBench [16] provides a multi-environment evaluation suite across operating systems, databases, and web tasks. These benchmarks demonstrate the value of stateful, multi-step evaluation protocols for agentic systems. ClinProg-Bench adapts this philosophy to the regulated software domain: its stateful runner contract (§4) supports multi-agent systems that can read specifications, write programs, inspect logs, and iterate—mirroring the real clinical-programming workflow. Unlike general agentic benchmarks, ClinProg-Bench evaluates domain-specific competence with deterministic oracles, avoiding the reliability and reproducibility concerns of LLM-as-a-judge scoring.

3 Dataset

3.1 Substrate and provenance

All tasks are grounded in the publicly redistributable CDISC SDTM/ADaM Pilot Project (CC-BY-4.0), pinned via Git submodule at commit 658fcc05506b169a27dee6e2c3a1ccdaaf64a716. Each task’s provenance block records its derivation script and the substrate commit, making generation end-to-end auditable. A SHA-256 manifest of 272 fixture files is committed alongside the tasks.

Vendored fixture subset. For grounding-conditioned baselines (§5), a 26-file, 682 KB subset of the substrate is vendored verbatim under `fixtures/`. The subset is the intersection of the 67 fixture base paths referenced by `inputs.fixtures` across the 250 task specifications and the files that exist in the v1.0.0 substrate; the remaining 42 references (15 SDTM domain-creator R programs and 27 TLF mock templates with hyphen/underscore naming drift) are unauthored upstream and are tracked as v0.1.1 work (§6). Vendoring is verbatim with no modification; substrate `LICENSE` and `NOTICE.md` accompany the files.

3.2 Composition

ClinProg-Bench contains **250** tasks distributed across five categories (Table 1). Each task is a validated TaskSpec JSON file bundled with fixture files, a markdown specification, and a gold output under `gold/<task_id>/`.

Table 1: Task taxonomy. Output kinds and scorers are fixed per sub-category to support deterministic evaluation.

Category	Sub-categories	N	Output kind	Scorer
T1 Code Generation	SDTM (22), ADaM (18), TLF (20)	60	program	codegen
T2 Code Review	program (25), validation (25)	50	JSON	log_review
T3 Spec Extraction	SDTM (21), ADaM (18), TLF (11)	50	JSON	spec_extract
T4 Documentation	TLF (16), dataset card (13), synopsis (11)	40	text	doc
T5 Debugging	SDTM (17), ADaM (17), TLF (16)	50	patch	debug
Total		250		

3.3 Quality assurance

Schema validation. Every task is parsed by the strict pydantic schema in `src/clinprog_bench/schema.py`; `task_id` is fixed to the regex `T[1-5]\.[1-9][0-9]?\. [a-z0-9_]+\.[0-9]{3}` and `seed_commit` to a 40-char SHA-1. Validation runs on every CI push (`.github/workflows/eval.yml`, `job validate`).

Leakage audit. A two-channel audit (`scripts/leakage_audit.py`) checks (a) SHA-256 byte overlap between gold outputs and the substrate, and (b) 13-gram Jaccard similarity between task prompts and substrate text. Both channels report **0 hits** on the v0.1.0 release; the report is committed at `docs/leakage-audit.md`.

122 **Dual-reviewer audit.** A stratified random subset (seed 20260426, 4 tasks per category, 20 total)
 123 is independently rated by two domain experts using a 5-point rubric. The acceptance threshold is
 124 Cohen’s $\kappa \geq 0.8$. Materials are shipped under `review_package/` and the audit log template is at
 125 `docs/audit-log.md`.

126 3.4 Licensing and release

127 The benchmark toolkit (Python package, scorers, runner contract) is released under **Apache-2.0**. The
 128 task corpus (JSON, gold outputs, fixtures) is released under **CC-BY-4.0**, inheriting the substrate
 129 license. A versioned snapshot is archived on Zenodo (DOI withheld for double-blind review). A full
 130 Gebru-style data card lives at `DATA_CARD.md` in the repository root.

131 4 Benchmark Design

132 4.1 Runner contract

133 ClinProg-Bench specifies a language-agnostic *runner contract*
 134 (`src/clinprog_bench/runners/contract.md`, v0.1.0) that any agent must satisfy. For
 135 each task the harness:

- 136 1. materialises a fresh workspace populated with the task’s fixtures,
- 137 2. invokes the agent’s `BaseAdapter.run(task, workspace)` method,
- 138 3. collects a `Submission` record (immutable; outputs keyed by the task’s `output_kind`),
- 139 4. passes the submission to the category-specific scorer.

140 The contract is intentionally *stateful*: the workspace persists across sub-tasks within a multi-step task,
 141 and the contract admits multi-agent and retrieval-augmented systems alongside single-turn LLMs.

142 4.2 Scorers

143 Each category has a single deterministic scorer; *no* LLM-as-a-judge is used. Scorers are pure
 144 functions of (gold output, submission), implemented in `src/clinprog_bench/scorers/`, and
 145 unit-tested (`tests/scorers/`, 79 tests, 100% pass on v0.1.0).

Table 2: Scoring rubrics per category.

Scorer	Method	Output range
<code>codegen</code>	AST-equivalence + slot-fill on key statements	<code>[0, 1]</code>
<code>log_review</code>	set match / confusion matrix on review findings	<code>[0, 1]</code>
<code>spec_extract</code>	set match on extracted slots	<code>[0, 1]</code>
<code>doc</code>	slot-fill on required documentation fields	<code>[0, 1]</code>
<code>debug</code>	patch-apply: does the patch fix the seeded defect?	<code>{0, 1}</code>

146 The aggregate benchmark score is the macro-average across the five categories, reported alongside
 147 per-category and per-sub-category breakdowns.

148 4.3 Reference baseline

149 A rules-only `ReferenceBaselineAdapter` (`src/clinprog_bench/runners/baselines/reference.py`)
 150 emits empty or trivially-formed submissions. Its purpose is *not* comparison but scorer validation: the
 151 reference baseline scores 0.00 across all categories, confirming that no scorer admits trivial outputs.

152 4.4 Evaluation harness and CI

153 The benchmark CI workflow (`.github/workflows/eval.yml`) runs three jobs on every push to
 154 master: schema-lint, unit tests, and a reference-baseline smoke evaluation. A successful CI run is a
 155 precondition for tagging a release.

5 Baselines

5.1 Setup

We evaluate two classes of baselines under the runner contract of §4:

1. **Rules-only reference** (reference-baseline): a deterministic adapter that emits trivially-formed submissions, run on all 250 tasks. It validates that no scorer admits empty answers and establishes the deterministic floor.
2. **Single-turn LLM** (deepseek-chat, claude-sonnet-4.5, glm-4.5): the task prompt and category-specific output instructions are passed to a frontier model in one call; the parsed response is scored as a submission. Each model is run under two grounding conditions: (i) *zero-shot*, prompt only; (ii) *+grounding*, with vendored CDISC fixtures (§3) inlined into the prompt under an 8 KB-per-request budget.

A retrieval-augmented agent (BM25 over the same fixture subset) is deferred to v0.1.1 and discussed in §6.

5.2 Decoding and budget

All LLM baselines use temperature 0, top_p= 1, and a 4096-token generation cap. Each LLM is evaluated on a stratified subset of 25 tasks (5 per category, seed 20260427); the reference baseline runs on all 250. Wall-clock latency for the 25-task pass with grounding was 422 s for DeepSeek-Chat, 519 s for Claude Sonnet 4.5, and 1325 s for GLM-4.5; GLM-4.5 incurred 7 upstream read timeouts on long-prompt tasks that were scored as zero-output submissions.

5.3 Results

Table 3 reports the macro-average score per baseline under both conditions; per-task submissions and the live HTML leaderboard are at docs/leaderboard/.

Table 3: Baseline results on ClinProg-Bench v0.1.0. Per-category columns are reported for the *+grounding* condition. “ZS” denotes the zero-shot macro for comparison. LLM rows are evaluated on a stratified 25-task subset (5 per category, seed 20260427); the reference row covers all 250 tasks. $\Delta = +\text{grounding} - \text{zero-shot}$.

Baseline	T1	T2	T3	T4	T5	Macro	ZS	Δ
reference-baseline [†]	—	—	—	—	—	—	0.05	—
deepseek-chat	0.35	0.00	0.44	0.56	0.05	0.28	0.27	+0.01
claude-sonnet-4.5	0.25	0.00	0.44	0.56	0.00	0.25	0.26	−0.01
glm-4.5	0.10	0.00	0.08	0.44	0.07	0.14 [†]	0.22	−0.08 [†]

[†] Includes 7 upstream read timeouts on long-prompt tasks; the operational drop reflects provider-side context handling under fixture inlining, not raw capability.

[‡] Reference baseline emits trivially-formed submissions and does not consume fixtures; only the zero-shot column applies. Per-category zero-shot scores are T1 = 0.25, T2 = 0, T3 = 0, T4 = 0, T5 = 0.

5.4 Discussion

Four patterns are visible in Table 3. First, the category ordering is stable across grounding conditions: T4 (documentation) and T3 (specification extraction) are the easiest, T1 (code generation) is harder, and T2 (log review) and T5 (debug patches) remain near 0 even with grounding—these categories require the specific SAS-log and _v2 program fixtures that the v0.1.0 substrate does not yet author (§3). Second, frontier models cluster within roughly 5 macro-points of one another in the zero-shot setting (0.22–0.27), suggesting that prompt engineering and grounding budgets—not raw capability—dominate at v0.1.0. Third, fixture grounding does *not* uniformly help: DeepSeek-Chat gains +0.01 (T1 lifts from 0.20 to 0.35 once spec documents are inlined), Claude Sonnet 4.5 is unchanged within noise, and GLM-4.5 loses 0.08 macro-points operationally because longer prompts push tail latencies past the provider’s read-timeout. We report the operational number rather than retrying; see §6 for

189 the implications and the planned mitigation. Fourth, the identical T4 score (0.56) across DeepSeek
190 and Claude is a scorer ceiling, not chance: the documentation rubric awards partial credit for length
191 and citation structure that any competent LLM satisfies, but the strict “pass” threshold requires
192 evidence-grounded claims that none of the three models produces under either condition. Closing
193 this gap is the focus of the retrieval-augmented baseline planned for v0.1.1.

194 6 Limitations

195 **One substrate.** ClinProg-Bench v0.1.0 is grounded entirely in the CDISC SDTM/ADaM Pilot
196 Project (a phase-II Alzheimer’s-disease protocol). This bounds external validity: results on Pilot01-
197 derived tasks may not generalise across therapeutic areas, study phases, or proprietary corporate
198 standards. A v0.2 release with at least one additional public substrate is planned.

199 **Scorer rigidity.** Deterministic scorers reject semantically equivalent but syntactically divergent
200 answers (e.g. a SAS IF/THEN that achieves the same derivation as a WHERE clause). Slot-fill and AST-
201 equivalence rubrics mitigate but do not eliminate this. We recommend reporting scorer-conformance
202 error rates alongside the headline number.

203 **Static gold outputs.** Gold outputs are frozen at substrate commit 658f cc05; downstream regulatory
204 practice (CDISC versions, FDA guidance) evolves. The benchmark is versioned and Zenodo-archived
205 to make this drift explicit.

206 **Single English locale.** All prompts and specifications are in English. Multilingual clinical-
207 programming evaluation is left to future work.

208 **Partial fixture coverage.** 42 of the 67 fixture base paths referenced by task specifications point at
209 files unauthored in the v1.0 substrate (15 SDTM `create_*.R` programs for the CM, DS, EX, LB,
210 MH, QS, SC, SV domains, and 27 TLF mock templates including hyphen/underscore naming-style
211 drift). The runner silently skips unresolvable paths, so the no-grounding baseline is unaffected, but
212 the +grounding macro is upper-bounded by a 40% coverage gap on T2/T5. Authoring these fixtures
213 and a BM25 retrieval-augmented baseline are the headline items for v0.1.1.

214 **Operational variability under grounding.** GLM-4.5 with fixture inlining incurred 7 upstream
215 read-timeout errors out of 25 prompts, dropping its operational macro by 0.08 (§5). The benchmark
216 records the operational outcome rather than retrying because submission throughput under longer
217 prompts is itself a property worth measuring; submitters should report timeouts and retries explicitly.

218 7 Broader Impact and Ethics

219 **No human-subjects data.** All fixtures derive from the CDISC SDTM/ADaM Pilot Project, a public
220 reference dataset distributed by CDISC for educational use. The dataset contains no real patient
221 records, no protected health information, and no proprietary trial data. No IRB review is required.

222 **Intended use.** ClinProg-Bench is intended to evaluate code-LLMs and agentic systems on clinical-
223 programming tasks under reproducible conditions. It is *not* a certification of fitness for regulatory
224 submission: a model scoring well on this benchmark is not thereby cleared for production use in
225 a phase-III trial. Practitioners should continue to follow ICH E6 (R3), 21 CFR Part 11, and their
226 organisation’s validation SOPs.

227 **Misuse considerations.** The benchmark could in principle be abused to over-claim AI capability in
228 regulatory contexts. We mitigate this with: (a) explicit benchmark-vs-deployment scoping in this
229 section, (b) a deterministic, fully-public evaluation harness that resists cherry-picked reporting, and
230 (c) a leakage audit and dual-reviewer audit committed to the repository.

231 **Author qualifications and review.** The dataset was authored by the listed author with the CDISC
232 Pilot Project as substrate; a three-member expert panel (clinical statistical programmers in the listed
233 acknowledgements) audits a stratified subset under a Cohen’s $\kappa \geq 0.8$ acceptance criterion (§3).

234 **Carbon footprint.** v0.1 LLM baseline runs (§5) total under 5 M tokens and complete in well under
235 one GPU-hour-equivalent; the benchmark itself adds negligible compute cost. Future evaluations will
236 report token counts and wallclock budgets in the leaderboard.

237 **Maintenance.** The author commits to maintaining the benchmark through the next two NeurIPS
238 review cycles. Release cadence, deprecation policy, and contact channels are documented in the
239 repository README.md; the Zenodo DOI provides long-term archival.

240 References

- 241 [1] Jacob Austin, Augustus Odena, Maxwell Nye, et al. Program synthesis with large language
242 models. *arXiv preprint arXiv:2108.07732*, 2021.
- 243 [2] CDISC. CDISC SDTM implementation guide v3.4. [https://www.cdisc.org/standards/](https://www.cdisc.org/standards/foundational/sdtmig)
244 [foundational/sdtmig](https://www.cdisc.org/standards/foundational/sdtmig), 2023. Accessed 2026-04.
- 245 [3] CDISC. CDISC ADaM implementation guide v1.3. [https://www.cdisc.org/standards/](https://www.cdisc.org/standards/foundational/adam)
246 [foundational/adam](https://www.cdisc.org/standards/foundational/adam), 2024. Accessed 2026-04.
- 247 [4] Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on
248 code. *arXiv preprint arXiv:2107.03374*, 2021.
- 249 [5] Yangruibo Ding et al. CrossCodeEval: A diverse and multilingual benchmark for cross-file
250 code completion. *NeurIPS*, 2023.
- 251 [6] Scott L. Fleming et al. MedAlign: A clinician-generated dataset for instruction following with
252 electronic medical records. *AAAI*, 2024.
- 253 [7] Alex Gu et al. CRUXEval: A benchmark for code reasoning, understanding, and execution. In
254 *ICLR*, 2024.
- 255 [8] Dan Hendrycks, Steven Basart, Saurav Kadavath, et al. Measuring coding challenge competence
256 with APPS. In *NeurIPS Datasets and Benchmarks*, 2021.
- 257 [9] International Council for Harmonisation. ICH harmonised guideline: Integrated addendum
258 to E6(R1): Guideline for good clinical practice E6(R2). [https://www.ich.org/page/](https://www.ich.org/page/efficacy-guidelines)
259 [efficacy-guidelines](https://www.ich.org/page/efficacy-guidelines), 2016. Accessed 2026-04.
- 260 [10] Carlos E. Jimenez, John Yang, Alexander Wettig, et al. SWE-bench: Can language models
261 resolve real-world GitHub issues? In *ICLR*, 2024.
- 262 [11] Di Jin, Eileen Pan, Nassim Oufattole, et al. What disease does this patient have? a large-scale
263 open domain question answering dataset from medical exams. *Applied Sciences*, 2021.
- 264 [12] Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William Cohen, and Xinghua Lu. PubMedQA: A
265 dataset for biomedical research question answering. In *EMNLP*, 2019.
- 266 [13] Yuhang Lai, Chengxi Li, Yiming Wang, et al. DS-1000: A natural and reliable benchmark for
267 data science code generation. In *ICML*, 2023.
- 268 [14] Changshu Liu et al. LiveCodeBench: Holistic and contamination free evaluation of code large
269 language models. *arXiv preprint arXiv:2403.07974*, 2024.
- 270 [15] Tianyang Liu, Canwen Xu, and Julian McAuley. RepoBench: Benchmarking repository-level
271 code auto-completion systems. *arXiv preprint arXiv:2306.03091*, 2024.
- 272 [16] Xiao Liu et al. AgentBench: Evaluating large language models as agents. In *ICLR*, 2024.
- 273 [17] Grégoire Mialon, Clémentine Fourrier, Craig Swift, et al. GAIA: A benchmark for general AI
274 assistants. *arXiv preprint arXiv:2311.12983*, 2023.
- 275 [18] Tianbao Xie et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real
276 computer environments. *NeurIPS*, 2024.

- 277 [19] John Yang et al. SWE-agent: Agent–computer interfaces enable automated software engineering.
278 *arXiv preprint arXiv:2405.15793*, 2024.
- 279 [20] Tao Yu, Rui Zhang, Kai Yang, et al. Spider: A large-scale human-labeled dataset for complex
280 and cross-domain semantic parsing and text-to-SQL task. In *EMNLP*, 2018.
- 281 [21] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, et al. WebArena: A realistic web environment
282 for building autonomous agents. In *ICLR*, 2024.
- 283 [22] Terry Yue Zhuo et al. BigCodeBench: Benchmarking code generation with diverse function
284 calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024.

285 **NeurIPS Paper Checklist**

286 **1. Claims**

287 Question: Do the main claims made in the abstract and introduction accurately reflect the
288 paper’s contributions and scope?

289 Answer: [\[Yes\]](#)

290 Justification: Abstract and introduction clearly state the 250-task, 5-category benchmark
291 scope and the three-model baseline evaluation (§5).

292 **2. Limitations**

293 Question: Does the paper discuss the limitations of the work performed by the authors?

294 Answer: [\[Yes\]](#)

295 Justification: See §6.

296 **3. Theory assumptions and proofs**

297 Question: For each theoretical result, does the paper provide the full set of assumptions and
298 a complete (and correct) proof?

299 Answer: [\[N/A\]](#)

300 Justification: This is a benchmark/dataset paper with no theoretical results.

301 **4. Experimental result reproducibility**

302 Question: Does the paper fully disclose all the information needed to reproduce the main ex-
303 perimental results of the paper to the extent that it affects the main claims and/or conclusions
304 of the paper (regardless of whether the code and data are provided or not)?

305 Answer: [\[Yes\]](#)

306 Justification: Appendix A provides one-command repro instructions. Code, data, and CI
307 workflow included in supplementary.

308 **5. Open access to data and code**

309 Question: Does the paper provide open access to the data and code, with sufficient instruc-
310 tions to faithfully reproduce the main experimental results, as described in supplemental
311 material?

312 Answer: [\[Yes\]](#)

313 Justification: Full benchmark (250 tasks, gold outputs, 5 scorers, runner contract, CI
314 workflow) released as supplementary ZIP. Code: Apache-2.0. Data: CC-BY-4.0. Zenodo
315 DOI for persistence (withheld for double-blind review).

316 **6. Experimental setting/details**

317 Question: Does the paper specify all the training and test details (e.g., data splits, hyperpa-
318 rameters, how they were chosen, type of optimizer) necessary to understand the results?

319 Answer: [\[Yes\]](#)

320 Justification: §5 reports temperature, top_p, token cap, stratified subset seed, and wall-clock
321 latency.

322 **7. Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: All baselines use deterministic decoding (temperature 0). No stochasticity to report; results are exactly reproducible.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Appendix A reports hardware (Intel i7-12700H, 32 GB, no GPU), per-baseline API costs, and wall-clock time.

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: See §7. No human subjects; substrate is the public CDISC Pilot Project with synthetic patients.

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: §7 discusses potential for automation of safety-critical clinical programming and associated risks.

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The benchmark contains no pre-trained models, no human data, and no sensitive content. The substrate is publicly available synthetic clinical trial data.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The CDISC Pilot Project is cited and its CC-BY-4.0 license respected. Our derived corpus inherits CC-BY-4.0 (§3).

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: DATA_CARD.md (Gebru-style datasheet), croissant.json (MLCommons Croissant with core and RAI fields), CITATION.cff, and dual licenses (LICENSE, LICENSE-DATA) all committed alongside the asset.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

372 Answer: [N/A]
373 Justification: No crowdsourcing or human subjects research. The expert audit was performed
374 by volunteer domain experts reviewing benchmark task quality.

375 **15. Institutional review board (IRB) approvals or equivalent for research with human**
376 **subjects**

377 Question: Does the paper describe potential risks incurred by study participants, whether
378 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
379 approvals (or an equivalent approval/review based on the requirements of your country or
380 institution) were obtained?

381 Answer: [N/A]
382 Justification: No human subjects data; substrate is the public CDISC Pilot Project with
383 synthetic patients. No IRB required.

384 **16. Declaration of LLM usage**

385 Question: Does the paper describe the usage of LLMs if it is an important, original, or
386 non-standard component of the core methods in this research? Note that if the LLM is used
387 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
388 scientific rigor, or originality of the research, declaration is not required.

389 Answer: [Yes]
390 Justification: Three frontier LLMs (DeepSeek-Chat, Claude Sonnet 4.5, GLM-4.5) are
391 evaluated as baselines (§5). Their versions, decoding parameters, and API costs are fully
392 reported.

393 A Reproducibility

394 This appendix provides one-command instructions for reproducing the baseline experiments reported
395 in Table 3.

396 A.1 Environment setup

```
397 # Clone and install (Python >= 3.11)
398 git clone https://github.com/anonymous/clinprog-bench.git
399 cd clinprog-bench
400 pip install .                # or: uv sync
401
402 # Verify: schema validation + unit tests
403 clinprog-bench validate tasks/
404 pytest -q                   # 79 tests, <1s
```

405 A.2 Reference baseline (all 250 tasks)

```
406 python -m clinprog_bench.runners.reference_baseline
407 # Output: results/reference_baseline_<timestamp>.json
408 # Expected macro zero-shot: 0.05
```

409 A.3 LLM baselines (25-task stratified subset)

410 Requires API keys set as environment variables: DEEPSEEK_API_KEY, ANTHROPIC_API_KEY,
411 GLM_API_KEY.

```
412 # Run all three LLM baselines (both conditions)
413 python scripts/run_llm_baseline.py \
414     --models deepseek-chat claude-sonnet-4-5 glm-4-5 \
415     --subset-seed 20260427 --subset-size 25 \
416     --conditions zero-shot grounding \
417     --output results/
418
419 # Results are written per-model to results/<model>_<condition>.json
```

420 **A.4 Continuous integration**

421 The GitHub Actions workflow (`.github/workflows/eval.yml`) runs on every push and pull
422 request. It executes three jobs sequentially: (1) **schema-lint**: validates all 250 task JSONs, runs
423 `ruff`, `black`, and `mypy`; (2) **unit**: runs the full `pytest` suite; (3) **baseline**: smoke-tests the reference
424 baseline runner. The CI badge in the repository README reflects the latest master status.

425 **A.5 Hardware and cost**

426 All baseline experiments were run on a consumer laptop (Intel i7-12700H, 32 GB RAM, no GPU)
427 with API calls to external model providers. Per-baseline API costs were approximately: DeepSeek-
428 Chat \$0.12 (25 tasks $\times$ 2 conditions), Claude Sonnet 4.5 \$1.80, GLM-4.5 \$0.08. Total wall-clock
429 time: $\sim$ 40 minutes for the full 25-task three-model matrix.